UNIVERSIDAD NACIONAL DE COLOMBIA

FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS E INDUSTRIAL
LENGUAJES DE PROGRAMACIÓN

TALLER # 01 — ANÁLISIS LÉXICO LEX: SISTEMA DE PROCESAMIENTO DEL LENGUAJE (SPL) PARA ENIGMA-64

AUTORES:

Juan Sebastian Umaña Camacho
Tomas Felipe Garzón Gómez
Michael Stiven Betancourt Gelves
Deibyd Santiago Barragán Gaitán
Maicol Sebastian Olarte Ramirez
Juan Luis Vergara Novoa
Alejandro Argüello Muñoz

Docente: Jorge Eduardo Ortiz Triviño

Bogotá D.C., Colombia — Octubre de 2026

CONTENIDO

Sección	Título
—	Enunciado y equipo de trabajo
1	Marco teórico
2	Descripción y justificación del problema
3	Diseño de la solución (arquitectura, diagrama de clases y convenciones)
4	Literal 2.a: Preprocesador
5	Literal 2.b: Ensamblador (analizador léxico)
6	Literal 2.c: Enlazador-Cargador
7	Literal 2.d: Módulo de análisis léxico del compilador (i, ii, iii)
8	Literal 2.e: Herramientas, metalenguaje y versiones
9	Manual técnico
10	Manual de usuario
11	Experimentación y análisis de resultados
12	Limitaciones conocidas y trabajo para el Taller # 02
13	Conclusiones
—	Referencias
A	Anexo: inventario del código fuente

Correspondencia con los entregables de una tarea automatizada.

El taller se entrega en un único archivo, como permite la observación a de los lineamientos del curso. La tabla indica dónde está cada entregable:

Entregable exigido	Ubicación en este documento
a. Marco teórico	Sección 1
b. Descripción y justificación del problema	Sección 2
c. Diseño de la aplicación (diagramas de bloques, de flujo, de estados y UML)	Secciones 3 a 7 (Diagramas 1 a 9)
d. Código fuente completo y versiones de las librerías	Sección 8 (versiones) y Anexo A (inventario); el código completo acompaña la entrega
e. Manual de usuario y manual técnico	Secciones 9 y 10
f. Experimentación y análisis de al menos 3 escenarios	Sección 11 (3 escenarios integrales y 2 complementarios)
Información de todos los autores	Portada y tabla de roles del equipo

ENUNCIADO

Análisis, diseño e implementación del Sistema de Procesamiento del Lenguaje SPL (Preprocesador, Compilador, Ensamblador y Enlazador-Cargador) para el computador diseñado e implementado en la solución de las Tareas # 09 y # 10 de la asignatura. El compilador debe construirse bajo el paradigma imperativo con tipos de datos abstractos (que incluya, cuando menos, las estructuras de secuencia, selección, iteración y TDA), con la posibilidad de definir y manejar variables, arreglos y matrices y con capacidad de realizar aritmética de punto flotante. La primera fase, objeto de este Taller # 01, consiste en:

a. Análisis, diseño e implementación del PREPROCESADOR empleando el metalenguaje FLEX (debe gestionar, cuando menos, las macros de incluir y definir).

b. Análisis, diseño e implementación del ENSAMBLADOR empleando el metalenguaje FLEX.

c. Análisis, diseño e implementación del ENLAZADOR-CARGADOR empleando el metalenguaje FLEX.

d. Análisis, diseño e implementación del MÓDULO DE ANÁLISIS LÉXICO DEL COMPILADOR para la máquina, empleando FLEX: (i) determinar el vocabulario completo que reconocería el compilador; (ii) definir las categorías gramaticales necesarias; (iii) especificar los patrones basados en expresiones regulares, definiciones y extensiones regulares para cada categoría léxica propuesta.

e. La construcción de todos estos módulos debe realizarse con FLEX junto con C y C++ (alternativamente, Java con JFLEX o Python con lex.py). Los demás entregables son, como mínimo, los de una tarea automatizada.

Equipo de trabajo y roles

El grupo es el mismo de las Tareas # 09 y # 10. La distribución de responsabilidades del taller fue la siguiente:

Integrante	Nombre	Módulo del SPL	Rol desempeñado
1	Juan Sebastian Umaña Camacho	Preprocesador	Lexer con estados del preprocesador; #define, #include, condicionales y supresión de comentarios.
2	Tomas Felipe Garzón Gómez	Ensamblador	Tokenizador del lenguaje ensamblador: 41 mnemónicos, registros, directivas, etiquetas y literales.
3	Michael Stiven Betancourt Gelves	Enlazador-Cargador	Formato .obj, análisis de módulos objeto, enlace de dos pasadas, reubicación y carga.
4	Juan Luis Vergara Novoa	Compilador: vocabulario	Diseño del lenguaje; palabras reservadas, tipos, TDA, arreglos, matrices y delimitadores.
5	Maicol Sebastian Olarte Ramirez	Compilador: operadores y literales	Operadores aritméticos, relacionales y lógicos; literales enteros, flotantes, cadenas y caracteres; validación de rangos.
6	Deibyd Santiago Barragán Gaitán	Tablas de símbolos	InstallID, InstallNum (enteros de 64 bits y reales IEEE 754) e InstallString, en Python y en ANSI C.
7	Alejandro Argüello Muñoz	Driver y pruebas	Orquestador del pipeline, verificación del módulo C con GCC y escenarios integrales de prueba.

1. MARCO TEÓRICO

1.1. Sistema de procesamiento del lenguaje

Un programa escrito en un lenguaje de alto nivel no se ejecuta directamente: atraviesa una cadena de traductores que Aho, Lam, Sethi y Ullman llaman sistema de procesamiento del lenguaje. El preprocesador reúne el programa fuente, que puede estar repartido en varios archivos, y expande las macros. El compilador traduce el programa preprocesado a lenguaje ensamblador. El ensamblador produce código máquina reubicable, organizado en módulos objeto. El enlazador combina varios módulos objeto, resuelve las referencias entre ellos y fija las direcciones definitivas; el cargador deposita el ejecutable en memoria y transfiere el control a su punto de entrada.

El compilador, a su vez, se divide en fases. La primera es el análisis léxico, que lee la secuencia de caracteres y la agrupa en unidades con significado; le siguen el análisis sintáctico, el semántico, la generación de código intermedio, la optimización y la generación de código. Este taller construye la fase léxica del compilador y los analizadores de las demás herramientas; las fases restantes corresponden al Taller # 02.

1.2. Análisis léxico: token, patrón, lexema y atributo

- **Token:** par formado por un nombre (la categoría gramatical, por ejemplo ID o RELOP) y un valor de atributo opcional. Es la unidad que recibe el analizador sintáctico.
- **Patrón:** descripción de la forma que pueden tomar los lexemas de un token; se escribe como expresión regular.
- **Lexema:** secuencia concreta de caracteres del programa fuente que coincide con el patrón de un token (en FLEX, yytext).
- **Atributo:** información adicional del token. En FLEX viaja en la variable global yylval: para un operador es un código (LT, LE, ...), y para un identificador o una constante es la posición del lexema en su tabla de símbolos.

Separar el análisis léxico del sintáctico simplifica ambos diseños, permite técnicas especializadas y eficientes de lectura de la entrada y aísla del resto del compilador las particularidades del alfabeto. El analizador léxico también elimina los espacios y los comentarios, lleva la cuenta de las líneas para los mensajes de error y detecta los errores léxicos: caracteres fuera del alfabeto, literales mal formados o cadenas sin cerrar.

1.3. Expresiones regulares, definiciones y extensiones regulares

Sobre un alfabeto Σ, las expresiones regulares se construyen a partir de ε y de los símbolos de Σ con tres operaciones: unión (r|s), concatenación (rs) y cerradura de Kleene (r*). Una definición regular asigna nombres a expresiones para reutilizarlas, siempre que cada nombre use solo los definidos antes que él:

```
letra   →  [A-Za-z_]
digito  →  [0-9]
id      →  letra ( letra | digito )*
expo    →  [eE] [+-]? digito+
```

Las extensiones regulares no aumentan el poder expresivo, pero abrevian la escritura: cerradura positiva r+ (= rr*), opcionalidad r? (= r|ε), clases de caracteres [a-z], clases negadas [^"\n] y repeticiones acotadas r{m,n}. FLEX añade además el contexto a la derecha r/s, que reconoce r solo si le sigue s; en Python, la herramienta lex.py ofrece el mismo efecto con las aserciones de anticipación (?=s) y (?!s).

1.4. Autómatas finitos y reglas de resolución de conflictos

Toda expresión regular puede convertirse en un autómata finito no determinista (AFN) mediante la construcción de Thompson, y este en un autómata finito determinista (AFD) mediante la construcción de subconjuntos; el AFD

resultante se minimiza y se codifica como una tabla de transiciones. Un generador de analizadores léxicos une los AFN de todos los patrones bajo un estado inicial común y resuelve los conflictos con dos reglas:

- **Coincidencia más larga:** se prefiere el prefijo más largo de la entrada que coincida con algún patrón; por eso <= se reconoce como un solo token y no como < seguido de =.
- **Prioridad por orden:** si dos patrones reconocen el mismo prefijo más largo, gana el que aparece primero en la especificación; así una palabra reservada listada antes que el patrón de identificadores se clasifica como palabra reservada.

1.5. FLEX y su equivalente en Python: lex.py (PLY)

Una especificación FLEX tiene tres secciones separadas por %%: definiciones (código C entre %{ y %} y definiciones regulares), reglas (pares patrón { acción }) y rutinas de usuario. FLEX genera lex.yy.c con la función yylex(), que devuelve el token reconocido y deja el lexema en yytext, su longitud en yyleng, la línea en yylineno y el atributo en yylval. Los estados de arranque exclusivos (%x) activan subconjuntos de reglas, por ejemplo dentro de un comentario de bloque.

El enunciado admite como alternativa Python con lex.py, el módulo léxico de PLY (Python Lex-Yacc, de David Beazley). El grupo eligió esta opción porque el emulador de Enigma-64 de la Tarea # 10 está escrito en Python, de modo que el SPL y la máquina comparten lenguaje, pruebas automatizadas y estructuras de datos. Los conceptos se corresponden uno a uno:

FLEX (C)	lex.py (PLY)
Sección de definiciones: digito [0-9]	Constantes de módulo DIGITO = r'[0-9]' combinadas con el decorador @TOKEN
Regla patrón { acción }	Método t_NOMBRE(self, t) cuya docstring o @TOKEN es el patrón; o atributo t_NOMBRE = r'...'
yytext, yyleng	t.value, len(t.value)
yylineno	t.lexer.lineno (se incrementa en la regla de salto de línea)
yylval	atributos añadidos al token: t.categoria_codigo, t.subcodigo, t.simbolo_idx
return TOKEN; / acción vacía	return t / return None (descartar)
%x estado, BEGIN(estado)	states = (('estado', 'exclusive'),) y t.lexer.begin('estado')
Regla . de error al final	t_error(self, t) con t.lexer.skip(1)
%option noyywrap	no aplica: lexer.input(texto) recibe la entrada completa
%option caseless	comparar palabra.upper() contra la tabla, o la bandera (?i:...)

Hay una diferencia que guía el diseño de todos los analizadores del grupo: PLY no aplica la regla de la coincidencia más larga entre reglas distintas. Construye una única expresión maestra con las reglas definidas como funciones, en el orden en que aparecen en el archivo, seguidas de las definidas como cadenas ordenadas por longitud decreciente de su expresión regular, y se queda con la primera alternativa que coincide. Por eso los patrones más específicos se escriben antes que los generales, y las palabras reservadas se reconocen con una sola regla de identificador seguida de una consulta a la tabla de palabras reservadas, técnica que recomienda el propio manual de PLY y que produce el mismo resultado que la coincidencia más larga de FLEX.

1.6. Tablas de símbolos

La tabla de símbolos guarda un registro por cada nombre o constante distinto del programa. El analizador léxico la consulta antes de insertar, de modo que cada lexema queda una sola vez (InstallID, InstallNum e InstallString en la Figura 3.18 de Aho et al.), y entrega como atributo la posición del registro. Las fases posteriores añaden a ese registro el tipo, el ámbito y la dirección asignada.

1.7. Ensamblador, enlazador y cargador

Un ensamblador de dos pasadas recorre el programa una vez para asignar una dirección a cada etiqueta (tabla de símbolos del ensamblador) y una segunda vez para traducir cada instrucción con las direcciones ya conocidas. Cuando un programa se divide en módulos, el ensamblador no conoce la dirección final de su código ni la de los símbolos externos: emite un módulo objeto reubicable con sus secciones, una tabla de símbolos exportados e importados y marcas de reubicación que indican qué campos deben corregirse. El enlazador asigna una dirección a cada sección, construye la tabla global de símbolos externos (ESTAB), verifica que toda importación tenga una definición y aplica las marcas. El cargador valida la región de destino, copia la imagen en memoria e inicializa el contador de programa con el punto de entrada (Beck, System Software).

1.8. Aritmética de punto flotante IEEE 754 binary64

El lenguaje debe manejar aritmética de punto flotante. El formato binary64 de la norma IEEE 754 usa 64 bits: 1 de signo, 11 de exponente con sesgo 1023 y 52 de fracción con un bit implícito. Su mayor valor finito es ≈ 1.7976931348623157 × 10^308, el menor normal positivo ≈ 2.2250738585072014 × 10^-308 y el menor subnormal ≈ 4.9 × 10^-324. Un literal decimal mayor que el máximo se redondea a infinito y uno con mantisa no nula menor que la mitad del menor subnormal se redondea a cero; el analizador léxico del compilador rechaza ambos casos porque el valor almacenado ya no representaría el literal escrito. La biblioteca de punto flotante de Enigma-64 (Tarea # 17) implementa este formato por software.

1.9. Máquina objetivo: el computador Enigma-64 (Tareas # 09 y # 10)

El SPL traduce para el computador Enigma-64 de Noctua Systems, diseñado en la Tarea # 09 y emulado en la Tarea # 10. Sus características condicionan el diseño léxico: los enteros caben en 64 bits, los reales son binary64, los registros tienen nombres fijos y los programas de usuario se cargan en una región determinada del mapa de memoria.

Característica	Valor
Arquitectura	Von Neumann: memoria unificada de datos e instrucciones
Palabra, buses de datos y de direcciones	64 bits
Unidad direccionable y ordenamiento	Byte; Big-Endian (el byte más significativo en la dirección menor)
Memoria física	4 GiB (0x00000000 – 0xFFFFFFFF), 8 bancos de 8 bits y alineación natural de los datos
Registros arquitectónicos	R0 (cableado a cero), R1–R4 de propósito general, R5 = RV (retorno), R6 = SP (pila), R7 = BP (marco), PC y SR (banderas Z, N, C, V)
Conjunto de instrucciones	41 instrucciones en 6 formatos de longitud variable (1 a 5 bytes); el opcode ocupa el primer byte y cada registro un nibble
Unidad de control	FSM de 5 fases (FETCH, DECODE, EXECUTE, MEMORY, WRITE-BACK) con cola de prebúsqueda de 16 bytes
E/S	Mapeada en memoria desde 0xFF000000, una página de 4 KiB por controlador

Mapa de memoria (Tarea # 09):

Región	Rango	Permisos
Vectores	`0x00000000 – 0x00000FFF`	R/X
Monitor + Enlazador-Cargador	`0x00001000 – 0x000FFFFF`	R/X
Trabajo del cargador	`0x00100000 – 0x0011FFFF`	R/W (solo modo administrador)
Tablas del sistema	`0x00120000 – 0x001FFFFF`	R/W (solo modo administrador)
Programas y datos de usuario	`0x00200000 – 0xBFFFFFFF`	R/W/X
Pila (crece hacia abajo)	`0xC0000000 – 0xEFFFFFFF`	R/W, no ejecutable
Buffers de E/S y DMA	`0xF0000000 – 0xFEFFFFFF`	R/W, no cacheable

Región	Rango	Permisos
E/S mapeada	`0xFF000000 – 0xFFFFFFFF`	R/W, no cacheable, administrador

2. DESCRIPCIÓN Y JUSTIFICACIÓN DEL PROBLEMA

Al terminar la Tarea # 10, Enigma-64 solo podía programarse escribiendo ensamblador o código máquina a mano. El problema del taller es dotarlo de la primera fase de un sistema de procesamiento del lenguaje: las herramientas que leen el texto de un programa de alto nivel, de un archivo en ensamblador o de un módulo objeto, y lo descomponen en unidades que las fases siguientes puedan traducir y combinar hasta obtener un ejecutable para la máquina.

2.1. Lenguaje de alto nivel propuesto

Se adopta el paradigma imperativo estructurado porque sus conceptos se corresponden directamente con los de la máquina: la asignación con LOAD y STORE, la secuencia con el avance del PC, la selección y la iteración con CMP y los saltos condicionales sobre las banderas del SR, y las funciones con CALL, RET, ENTER y LEAVE. El lenguaje ofrece:

- **Secuencia:** bloques { ... } y sentencias terminadas en punto y coma.
- **Selección:** if-then-else y switch-case-default con break.
- **Iteración:** while, do-while y for, con break y continue.
- **Subprogramas:** function y return; el valor de retorno viaja en RV = R5.
- **Tipos:** int (entero de 64 bits en complemento a dos), float (IEEE 754 binary64), char, bool, void y el calificador const.
- **TDA, arreglos y matrices:** struct y tda para agrupar campos, con acceso por . y ->; typedef y sizeof; arreglos v[n] y matrices m[f][c] dispuestos por filas, de modo que Dir(A[i][j]) = Base + (i·N + j)·8.
- **Punto flotante:** literales en punto fijo (3.14, .5, 5.) y en notación científica (6.022e23, 2.5E-3); las operaciones entre float se traducirán a las rutinas de la FPU por software de la Tarea # 17.

2.2. Justificación de las decisiones

- **Herramienta:** lex.py (PLY 3.11) es una de las alternativas que admite el enunciado. Mantiene en un solo lenguaje el SPL, el emulador y la suite de pruebas, y permite que el orquestador cargue el ejecutable directamente en la RAM y la CPU emuladas. El módulo de tablas de símbolos se escribió también en ANSI C para conservar la interfaz clásica de FLEX (InstallID, InstallNum, InstallString).
- **Rangos ligados a la máquina:** como los registros miden 64 bits, el analizador rechaza en la fase léxica los enteros que no caben en 64 bits y los reales que binary64 no puede representar, en lugar de truncarlos en silencio.
- **Tolerancia a errores:** todos los analizadores registran cada error con línea, columna y lexema, lo descartan y continúan, para informar todos los errores en una sola ejecución.
- **Atributos al estilo de la Tarea # 25:** cada token lleva un código de categoría (100 a 240) y un subcódigo o una posición en tabla, igual que el par token/yylval de los ejercicios de clase.

2.3. Alcance del Taller # 01

El taller cubre la fase léxica de las cuatro herramientas y el enlazador-cargador completo sobre módulos objeto. El análisis sintáctico y semántico, la generación de ensamblador y la emisión de módulos objeto por parte del ensamblador corresponden al Taller # 02; la Sección 12 detalla lo que queda pendiente.

3. DISEÑO DE LA SOLUCIÓN

3.1. Arquitectura general (diagrama de bloques)

El Diagrama 1 muestra el flujo de información. Los bloques azules están implementados en este taller; los grises corresponden al Taller # 02. El orquestador (python -m spl) ejecuta en secuencia las etapas implementadas y reúne sus métricas, tablas y errores en un único informe.

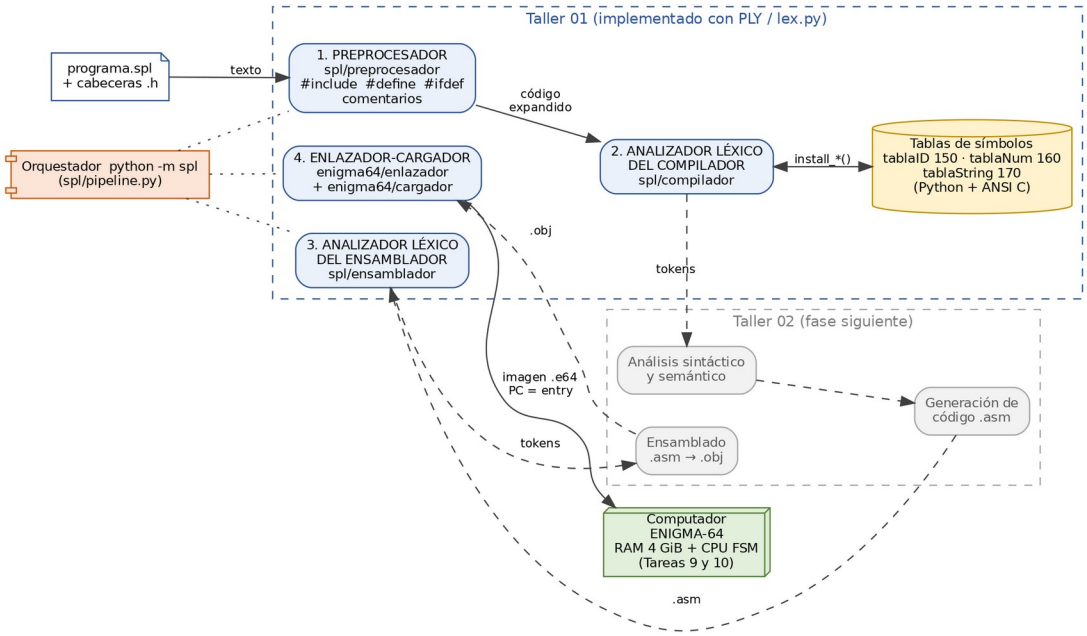


Diagrama 1: Arquitectura del SPL de Enigma-64 y fases de los Talleres # 01 y # 02.

Módulo	Paquete	Entrada	Salida
Preprocesador	`spl/preprocesador`	Archivo .spl y cabeceras .h	Código expandido, macros definidas y archivos incluidos
Analizador léxico del compilador	`spl/compilador`	Código expandido	Tokens con categoría y atributo; tablas ID, NUMBER y STRING; errores
Analizador léxico del ensamblador	`spl/ensamblador`	Archivo .asm	Tokens <línea, categoría, lexema>; errores
Enlazador-Cargador	`enigma64/enlazador`, `enigma64/cargador.py`	Módulos .obj	Ejecutable .e64 cargado en la RAM de Enigma-64 con PC = punto de entrada
Orquestador	`spl/pipeline.py`, `spl/__main__.py`	Escenario o archivos	Informe de las 4 etapas

3.2. Diagrama de clases (UML)

El Diagrama 2 resume las clases principales y sus relaciones. Cada analizador léxico es una clase que encapsula un lexer de PLY y ofrece la misma interfaz (input, token, tokenizar); el analizador del compilador agrega un GestorTablasSimbolos, y el enlazador agrega uno o más ModuloObjeto.

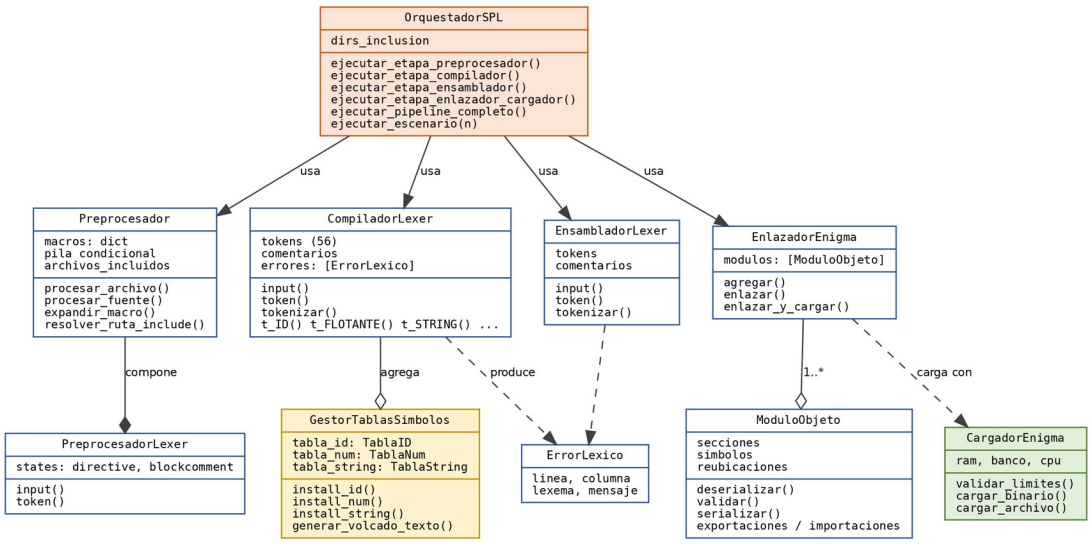


Diagrama 2: Diagrama de clases de los componentes del SPL.

3.3. Convenciones comunes a los analizadores

- **Token:** objeto LexToken de PLY con type (categoría), value (lexema), lineno y lexpos, al que los analizadores añaden columna y, en el compilador, categoria_codigo, subcodigo y simbolo_idx.
- **Error léxico:** registro ErrorLexico(linea, columna, lexema, mensaje). Se almacena en una lista, se puede notificar por un callback y no detiene el análisis.
- **Recuperación:** las reglas de error consumen el lexema completo cuando reconocen un literal mal formado (por ejemplo 0xFFG1) y un solo carácter cuando el carácter no pertenece al alfabeto (t_error con skip(1)).
- **Interfaz:** tokenizar(texto) devuelve la pareja (tokens, errores); los paquetes tienen además una interfaz de línea de comandos python -m <paquete>.

4. LITERAL 2.A: PREPROCESADOR

El preprocesador (`spl/preprocesador`) se compone de un analizador léxico con estados exclusivos escrito en lex.py (`lexer.py`) y de un motor (`motor.py`) que interpreta las directivas, mantiene la tabla de macros y resuelve las inclusiones.

4.1. Funcionalidades

Directiva / elemento	Comportamiento
`#include "archivo"`	Busca en el directorio del archivo actual, luego en los directorios -I y por último en spl/include/. Procesa el archivo incluido de forma recursiva.
`#include <archivo>`	Busca en los directorios -I y en spl/include/ (cabecera estándar enigma64.h).
`#define NOMBRE valor`	Registra una macro de objeto; sus apariciones posteriores fuera de cadenas y caracteres se sustituyen, y la expansión es recursiva con protección contra auto-referencias.
`#undef NOMBRE`	Elimina la macro.
`#ifdef / #ifndef / #else / #endif`	Compilación condicional mediante una pila de marcos; las líneas de una rama inactiva se descartan.
`#pragma once`	Evita incluir dos veces el mismo archivo.
`// y /* ... */`	Se suprimen; un comentario de bloque conserva los saltos de línea para no alterar la numeración.

Directiva / elemento	Comportamiento
`\ al final de línea`	Continuación de línea, también dentro de una directiva.
`-D NOMBRE=VALOR`	Macros predefinidas desde la línea de comandos.

4.2. Categorías léxicas y patrones

Estado	Token	Patrón (lex.py)
INITIAL	DIRECTIVE_START	`^[ \t]*\#   (entra al estado directive)`
INITIAL	COMMENT (línea)	`//[^\r\n]*`
INITIAL	inicio de bloque	`/\*   (entra al estado blockcomment)`
INITIAL	STRING	`\"([^\"\\\r\n]|\\.)*\"`
INITIAL	CHAR	`\'([^\'\\\r\n]|\\.)*\'`
INITIAL	NUMBER	`0[xX][0-9a-fA-F]+ | 0[bB][01]+ | [0-9]+(\.[0-9]+)?([eE][+-]?[0-9]+)? | \.[0-9]+...`
INITIAL	LINE_CONTINUATION	`\\([ \t]*\r?\n)`
INITIAL	IDENTIFIER	`[a-zA-Z_][a-zA-Z0-9_]*`
INITIAL	WHITESPACE / NEWLINE	`[ \t]+  /  \r?\n`
directive	DIR_DEFINE, DIR_INCLUDE, DIR_IFDEF, ...	`palabra clave de directiva tras el #`
directive	DIR_HEADER_SYS	`<[^>\r\n]*>`
directive	DIR_HEADER_LOCAL / DIR_STRING	`\"...\"`
directive	DIR_END	`\r?\n   (vuelve a INITIAL)`
blockcomment	fin de bloque	`\*/   (vuelve al estado previo)`

4.3. Diagrama de estados del analizador léxico

Los estados exclusivos equivalen a los %x de FLEX: dentro de una directiva o de un comentario de bloque solo están activas las reglas de ese estado (Diagrama 3).

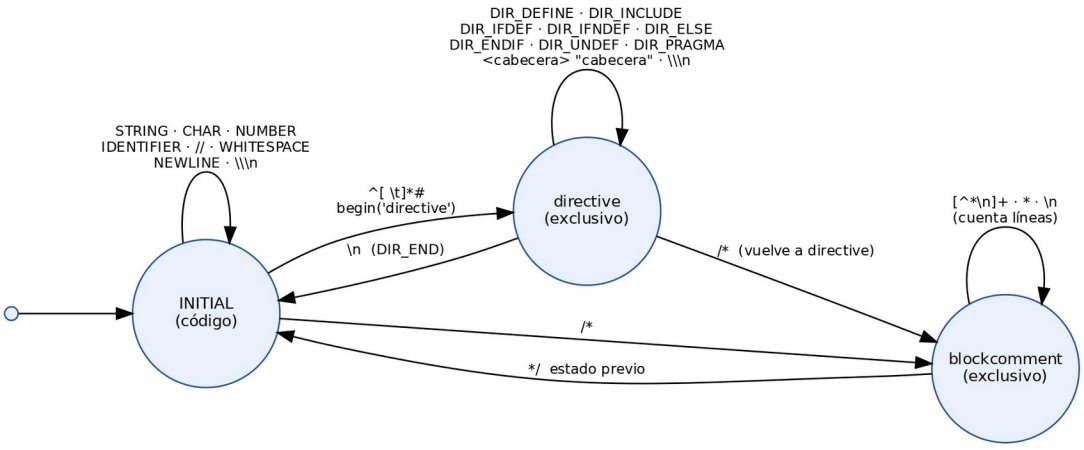


Diagrama 3: Estados léxicos del preprocesador (INITIAL, directive y blockcomment).

4.4. Algoritmo del motor (diagrama de flujo)

El Diagrama 4 muestra cómo el motor consume los tokens del lexer, ejecuta las directivas y expande las macros en las líneas activas.

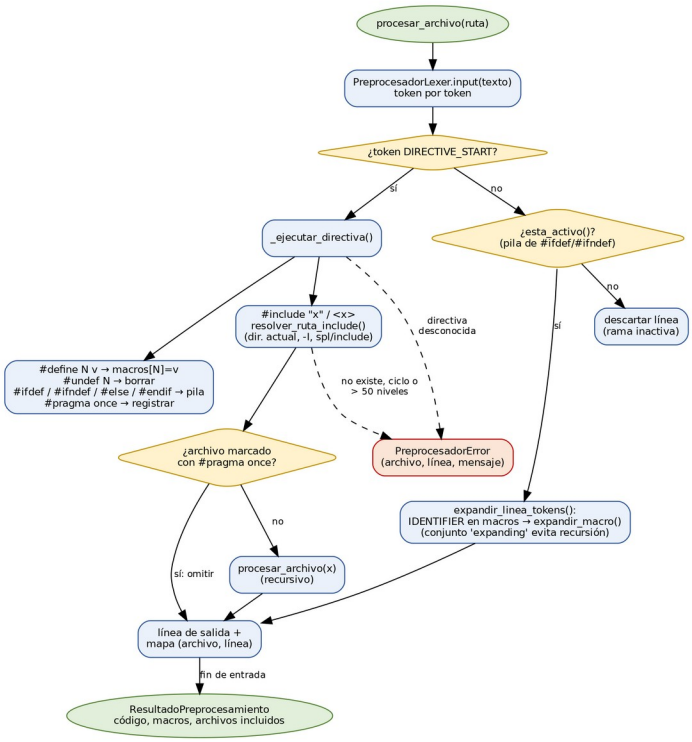


Diagrama 4: Procesamiento de un archivo: directivas, condicionales, inclusión y expansión de macros.

4.5. Errores detectados

El motor detiene el preprocesamiento con PreprocesadorError(archivo, línea, mensaje) ante: un archivo de inclusión inexistente; un ciclo de inclusión (se informa la cadena a -> b -> a); más de 50 niveles de inclusión anidada; un comentario de bloque sin cerrar; un #else o #endif sin directiva condicional previa, o un #else duplicado; un #define sin nombre de macro; una sintaxis inválida de #include; una directiva desconocida; y un #ifdef o #ifndef sin su #endif al final del archivo.

5. LITERAL 2.B: ENSAMBLADOR (ANALIZADOR LÉXICO)

El tokenizador del lenguaje ensamblador (`spl/ensamblador/lexer.py`) reconoce el vocabulario de Enigma-64. Las 41 instrucciones se tomaron de la tabla de decodificación de la CPU emulada (`enigma64/cpu.py`), y una prueba automatizada verifica que el lexer y la CPU tengan exactamente el mismo conjunto.

5.1. Vocabulario

Grupo	Elementos
Control	`HLT`
Aritmética entera	`ADD SUB MUL DIV ADDI SUBI INC DEC`
Lógica, corrimientos y comparación	`AND OR XOR NOT SHL SHR ASR CMP`
Memoria	`LOAD STORE LDB STB`
Saltos	`JMP JZ JNZ JC JNC JN JP JV JNV JMPR`
Subrutinas y pila	`CALL CALLR RET PUSH POP ENTER LEAVE`
Interrupciones	`EI DI IRET`
Registros	`R0–R7, PC, SR y los alias RV (R5), SP (R6), BP (R7)`
Directivas	`.data .text .org .word .byte`
Separadores y operadores	`, : [ ] ( ) # + -`

5.2. Categorías léxicas y patrones

```
DIGITO     [0-9]          HEX   [0-9a-fA-F]        BIN  [01]
LETRA      [a-zA-Z_]      ID    LETRA (LETRA|DIGITO)*
EXPONENTE  [eE][+-]?DIGITO+
FIN_NUM    (?![\w.])      el literal no puede ir pegado a letras, dígitos ni '.'
```

Categoría	Patrón	Ejemplos
MNEMONICO	`ID cuyo valor en mayúsculas está en la tabla de 41 instrucciones`	ADD, addi, Jmp
REGISTRO	`(?i: R[0-7] | PC | SR | RV | SP | BP )`	R0, r7, SP
DIRECTIVA	`\.ID que está en la tabla de directivas`	.data, .ORG
ETIQUETA_DEF	`ID(?=[ \t]*:)`	main en main:
ETIQUETA_REF	`ID que no es mnemónico ni registro`	loop en JMP loop
ENTERO_DEC	`-?DIGITO+ FIN_NUM`	16, -3
ENTERO_HEX	`-?0[xX]HEX+ FIN_NUM`	0x00201000
ENTERO_BIN	`-?0[bB]BIN+ FIN_NUM`	0b1010
FLOTANTE	`-?(DIGITO+\.DIGITO+(EXPONENTE)? | DIGITO+ EXPONENTE) FIN_NUM`	3.14159, -2.5e-3
COMA, DOS_PUNTOS, CORCHETE_*, PARENTESIS_*, INMEDIATO, MAS, MENOS	`,  :  [ ]  ( )  #  +  -`	[R1 + 8]
(ignorado)	`;[^\n]*  ·  [ \t\r]  ·  \n+ (cuenta líneas)`	; comentario

5.3. Resolución de conflictos

Como PLY prueba las reglas de función en orden, la especificación las dispone de la más específica a la más general (Diagrama 5). El signo menos tiene un tratamiento especial: forma parte del literal en ADDI R1, R0, -8, pero es el operador resta en [BP-8]; el lexer recuerda la categoría del último token y, si era un operando, emite MENOS y retrocede la posición de lectura.

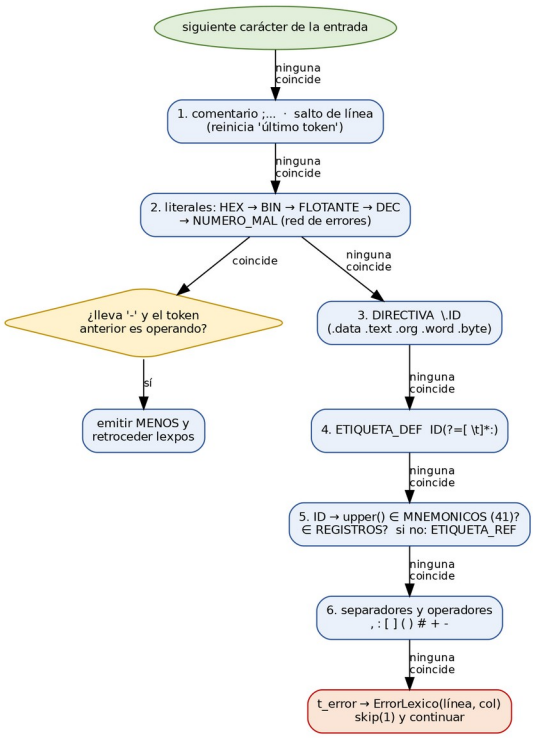


Diagrama 5: Orden de las reglas del analizador léxico del ensamblador.

5.4. Errores léxicos detectados

Error	Detección	Ejemplo
Carácter no válido	t_error: ninguna regla reconoce el carácter	`@ $ ?`
Literal numérico mal formado	-?DIGITO[\w.]* cuando ningún literal válido lo cubre	`0b102, 0xG1, 12abc`
Literal fuera de 64 bits	valor fuera de [−2^63, 2^64 − 1]	`0x1FFFFFFFFFFFFFFFF`
Directiva desconocida	\.ID que no está en la tabla	`.float`
Registro inexistente	[rR][0-9]+ que no es R0–R7	`R8, R10`
Palabra reservada como etiqueta	ETIQUETA_DEF cuyo nombre es mnemónico o registro	`LOAD:`

6. LITERAL 2.C: ENLAZADOR-CARGADOR

El enlazador-cargador (`enigma64/enlazador` y `enigma64/cargador.py`) ocupa, según la Tarea # 09, la región Monitor + Enlazador-Cargador (0x00001000 – 0x000FFFFF). Lee módulos objeto reubicables (.obj), los enlaza en un ejecutable absoluto (.e64) y lo carga en la RAM de Enigma-64.

6.1. Formato del módulo objeto (.obj)

El formato es binario y Big-Endian, como la máquina. Tras la firma "EOBJ" y la máquina 0xE064 vienen las tablas de secciones, símbolos, marcas de reubicación y cadenas:

```
+----------------------------------+ 0
| Cabecera                  52 B   |  magia EOBJ, versión, máquina 0xE064, entrada
+----------------------------------+ off_secciones
| Encabezados de sección    20 B c/u  nombre, tipo, permisos R/W/X, alineación
+----------------------------------+
| Contenido de secciones BITS      |  (.bss no ocupa bytes en el archivo)
+----------------------------------+ off_simbolos
| Tabla de símbolos         12 B c/u  LOCAL, GLOBAL (exportado) o importado
+----------------------------------+ off_reubicaciones
| Marcas de reubicación     16 B c/u  sección, desplazamiento, tipo, símbolo, sumando
+----------------------------------+ off_cadenas
| Tabla de cadenas (UTF-8 + NUL)   |
+----------------------------------+
```

6.2. Marcas de reubicación

Cada marca nace de un campo del ISA que contiene una dirección (S = dirección final del símbolo, A = sumando, P = dirección del campo parchado):

Tipo	Ancho	Fórmula	Uso en el ISA
ABS64	8 B	`S + A`	Palabra de datos que guarda una dirección (.word etiqueta)
ABS32	4 B	`S + A`	Destino de JMP y CALL (bytes 1 a 4)
REL16	2 B	`S + A − (P + 2)`	Desplazamiento de JZ, JNZ, ... hacia otro módulo
HI16	2 B	`(S + A + 0x8000) >> 16`	Inmediato del ADDI que precede a SHL 16
LO16	2 B	`(S + A) & 0xFFFF`	Inmediato de ADDI o desplazamiento de LOAD/STORE

HI16 es la parte alta ajustada: la CPU extiende el signo de los inmediatos de 16 bits, así que un LO16 ≥ 0x8000 resta 0x10000 y la parte alta lo compensa. Así, la secuencia ADDI Rx,R0,hi / SHL Rx,Rx,16 / ADDI Rx,Rx,lo construye la dirección exacta.

6.3. Algoritmo de enlace y carga

El Diagrama 6 resume el proceso; las líneas punteadas llevan a las excepciones que lo detienen.

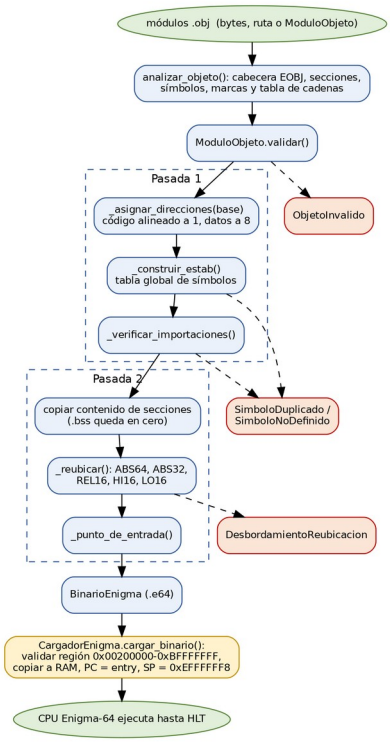


Diagrama 6: Enlace de dos pasadas, reubicación y carga en Enigma-64.

- **Pasada 1:** asigna direcciones a las secciones a partir de la base (0x00200000 por defecto), con alineación 1 para el código, porque las instrucciones son de longitud variable, y 8 para los datos, porque LOAD y STORE exigen alineación natural; construye la ESTAB y verifica que no haya símbolos duplicados ni importaciones sin definir.
- **Pasada 2:** copia el contenido de las secciones en la imagen, deja en cero las reservas .bss, aplica cada marca comprobando que el valor quepa en su campo y determina el punto de entrada (el símbolo indicado, el declarado por un módulo o el inicio del código).
- **Carga:** CargadorEnigma valida que la imagen quede en la región de usuario 0x00200000 – 0xBFFFFFFF, la copia en la RAM e inicializa PC con el punto de entrada y SP en la cima de la pila.

6.4. Nota sobre la técnica de análisis

Como el formato .obj es binario, el reconocimiento de sus campos se hace con registros de tamaño fijo (módulo struct de Python) y no con expresiones regulares: un campo de 4 bytes se identifica por su posición y no por su forma. Cada campo se valida (índices, desplazamientos, tamaños y nombres) y cualquier incoherencia produce ObjetoInvalido antes de tocar la RAM. La Sección 12 discute este punto frente a la exigencia de usar el metalenguaje.

7. LITERAL 2.D: MÓDULO DE ANÁLISIS LÉXICO DEL COMPILADOR

El analizador léxico del compilador (`spl/compilador/lexer.py`, clase CompiladorLexer) reconoce el lenguaje de alto nivel descrito en la Sección 2.1 y llena las tablas de símbolos (`spl/compilador/tablas.py` y su versión en C, `tablas_simbolos.c`).

7.1. (i) Vocabulario completo

El vocabulario es la unión disjunta V = V_control ∪ V_tipos ∪ V_tda ∪ V_id ∪ V_delim ∪ V_oper ∪ V_lit:

Conjunto	Elementos
V_control (13)	`if then else switch case default while do for continue function return break`
V_tipos (6)	`int float void char bool const`
V_tda (4)	`struct tda typedef sizeof`
V_id	`letra (letra | dígito)*, con letra = [A-Za-z_], excluyendo las palabras reservadas`
V_delim (12)	`[ ] { } ( ) ; , : . -> =`
V_oper relacionales (6)	`== != < <= > >=`
V_oper aritméticos (5)	`+ - * / %`
V_oper lógicos (3)	`&& || !`
V_lit enteros	`decimal (1000), hexadecimal (0xFF, 0X7FFF...), binario (0b1010)`
V_lit reales	`punto fijo (3.14159, .5, 5.) y notación científica (6.022e23, 2.5E-3, .5e+2)`
V_lit cadenas y caracteres	`"texto con \n \t \r \0 \\ \" \'" y 'a', '\n'`
Ignorados	`espacios, tabuladores, saltos de línea, // ... y /* ... */`

7.2. (ii) Categorías gramaticales y códigos

Cada token lleva un código de categoría y un subcódigo, como el par token/yylval de la Tarea # 25. Para ID, NUMBER y STRING el atributo es, además, la posición del lexema en su tabla:

Código	Categoría	Tokens (subcódigo)	Atributo
100	KW: control de flujo	IF 1, THEN 2, ELSE 3, SWITCH 4, CASE 5, DEFAULT 6, WHILE 7, DO 8, FOR 9, CONTINUE 10, FUNCTION 11, RETURN 12, BREAK 13	Subcódigo
110	TYPE: tipos primitivos	INT_TYPE 1, FLOAT_TYPE 2, VOID_TYPE 3, CHAR_TYPE 4, BOOL_TYPE 5, CONST 6	Subcódigo
120	TDA: tipos abstractos	STRUCT 1, TDA 2, TYPEDEF 3, SIZEOF 4	Subcódigo
150	ID: identificador	ID	Posición en tablaID
160	NUMBER: constante	ENTERO_DEC 1, ENTERO_HEX 2, ENTERO_BIN 3, FLOTANTE 4 (punto fijo) o 5 (científica)	Posición en tablaNum
170	STRING: cadena o carácter	STRING 1, CHAR_LITERAL 2	Posición en tablaString
200	RELOP: relacional	IGUAL 1, DISTINTO 2, MENOR 3, MENOR_IGUAL 4, MAYOR 5, MAYOR_IGUAL 6	Subcódigo
210	ARITH: aritmético	MAS 1, MENOS 2, POR 3, DIV 4, MOD 5	Subcódigo
220	LOGOP: lógico	AND_LOGICO 1, OR_LOGICO 2, NOT_LOGICO 3	Subcódigo
230	ASSIGN: asignación	ASIGNAR	—
240	DELIM: delimitador	CORCHETE_IZQ/DER, LLAVE_IZQ/DER, PARENTESIS_IZQ/DER, PUNTO_Y_COMA, COMA, DOS_PUNTOS, PUNTO, FLECHA	—

7.3. (iii) Definiciones regulares

```
DIGITO      [0-9]
HEX         [0-9a-fA-F]
BIN         [01]
LETRA       [a-zA-Z_]
FIN_NUM     (?![\w.])                          extensión: contexto derecho negativo
EXPONENTE   [eE][+-]?DIGITO+
MANTISA     DIGITO+\.DIGITO* | \.DIGITO+       3.14   5.   .5
ESCAPE      \\[ntr0\\"']
ID          LETRA (LETRA | DIGITO)*
ENTERO_HEX  0[xX]HEX+ FIN_NUM
ENTERO_BIN  0[bB]BIN+ FIN_NUM
FLOTANTE    ( MANTISA EXPONENTE? | DIGITO+ EXPONENTE ) FIN_NUM
ENTERO_DEC  DIGITO+ FIN_NUM
CADENA      "( [^"\\\n\r] | ESCAPE )*"
CARACTER    '( [^'\\\n\r] | ESCAPE )'
```

7.4. Patrones de los tokens y de los errores

La tabla sigue el orden de la especificación, que es el orden en que PLY prueba las reglas. Las reglas de error van inmediatamente después de la regla válida que extienden, de modo que una entrada solo cae en ellas si la regla válida no la reconoce:

Orden	Regla	Patrón	Resultado
1	Comentarios y saltos de línea	`//[^\n]*  /\*(.|\n)*?\*/  \n+`	Se descartan; se cuentan las líneas
2	Operadores de dos caracteres	`-> == != <= >= && ||`	FLECHA, RELOP, LOGOP
3	Delimitadores y operadores simples	`[ ] { } ( ) ; , : . = + - * / % < > !`	DELIM, ASSIGN, ARITH, RELOP, LOGOP
4	STRING	`CADENA`	Instala el texto des-escapado en tablaString
5	Error: escape no admitido	`"([^"\\\n\r]|\\[^\n\r])*"`	"... \q ..."
6	Error: cadena sin cerrar	`"([^"\\\n\r]|\\[^\n\r])*\\?`	Hasta el fin de línea
7	CHAR_LITERAL / error	`CARACTER / '([^'\\\n\r]|\\[^\n\r])*'?`	'a', '\n' / 'ab', '\q'
8	ENTERO_HEX, ENTERO_BIN	`0[xX]HEX+ FIN_NUM, 0[bB]BIN+ FIN_NUM`	Máximo 2^64 − 1 (patrón de 64 bits)
9	FLOTANTE	`FLOTANTE`	Rechaza ±inf y el subdesbordamiento a cero
10	ENTERO_DEC	`DIGITO+ FIN_NUM`	Máximo 2^63 − 1 (INT64_MAX)
11	Error: exponente incompleto	`(MANTISA|DIGITO+)[eE][+-](?!DIGITO)`	1e+, 2.5E-
12	Error: número mal formado	`(DIGITO|\.DIGITO)[\w.]*`	0xFFG1, 0b102, 45xyz, 1.2.3
13	ID / palabra reservada	`ID, luego consulta a PALABRAS_RESERVADAS`	KW/TYPE/TDA o ID en tablaID
14	Error: operador incompleto	`&(?!&) | |(?!|)`	& y | sueltos
15	Error: carácter no válido	`t_error (cualquier otro carácter)`	@ $ `

Las expresiones de dos caracteres se reconocen antes que las de uno porque PLY ordena las reglas de cadena por longitud decreciente de su patrón, lo que reproduce la coincidencia más larga para los operadores. El contexto derecho FIN_NUM evita que 0b102 se lea como 0b10 seguido de 2: el literal válido no coincide y la entrada completa cae en la regla de número mal formado.

7.5. Autómatas finitos de los literales

El Diagrama 7 muestra el AFD equivalente a los patrones numéricos (los estados de aceptación indican el subcódigo). La anticipación FIN_NUM, que no se dibuja, exige que el carácter siguiente no sea letra, dígito ni punto. El Diagrama 8 muestra el autómata de las cadenas y sus dos estados de error.

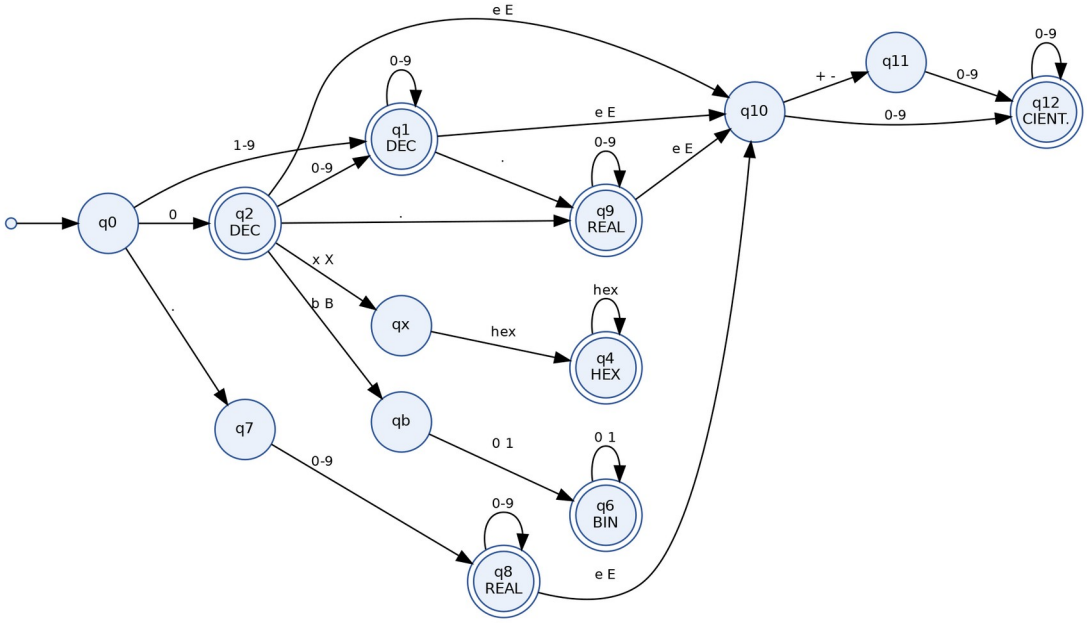


Diagrama 7: AFD de los literales numéricos: decimal, hexadecimal, binario, real en punto fijo y en notación científica.

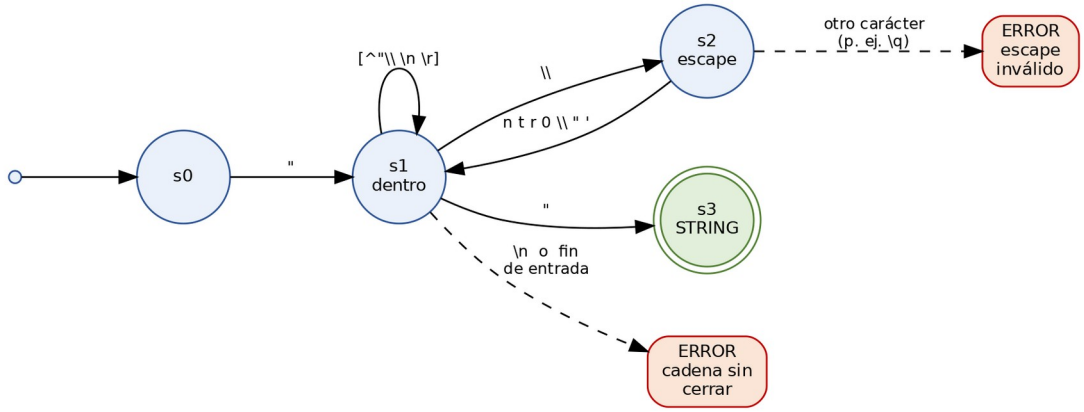


Diagrama 8: AFD de las cadenas con secuencias de escape y estados de error.

7.6. Validación de rangos

Literal	Rango aceptado	Motivo
Entero decimal	0 … 2^63 − 1	Un int de 64 bits con signo; el signo es el operador unario, de modo que −2^63 se escribe como expresión
Entero hexadecimal o binario	0 … 2^64 − 1	Representa un patrón de 64 bits (por ejemplo 0xFFFFFFFFFFFFFFFF)
Real	$\|x\| \leq 1.797\ldots e308$ y $x \neq 0$ si la mantisa no es nula	Fuera de ese rango binary64 devolvería ±inf o 0

7.7. Tablas de símbolos

Las reglas aceptoras de ID, de los literales numéricos y de STRING llaman al gestor de tablas, que busca el lexema antes de insertarlo (Diagrama 9). tablaNum guarda el formato, el valor entero de 64 bits y el valor real binary64; tablaString guarda el texto sin comillas y con los escapes ya interpretados, de modo que "a\tb" queda con un tabulador real. El mismo algoritmo está escrito en ANSI C11 (tablas_simbolos.h y tablas_simbolos.c) con su propia batería de pruebas (test_tablas.c), que se compila con GCC mediante el Makefile de spl/compilador.

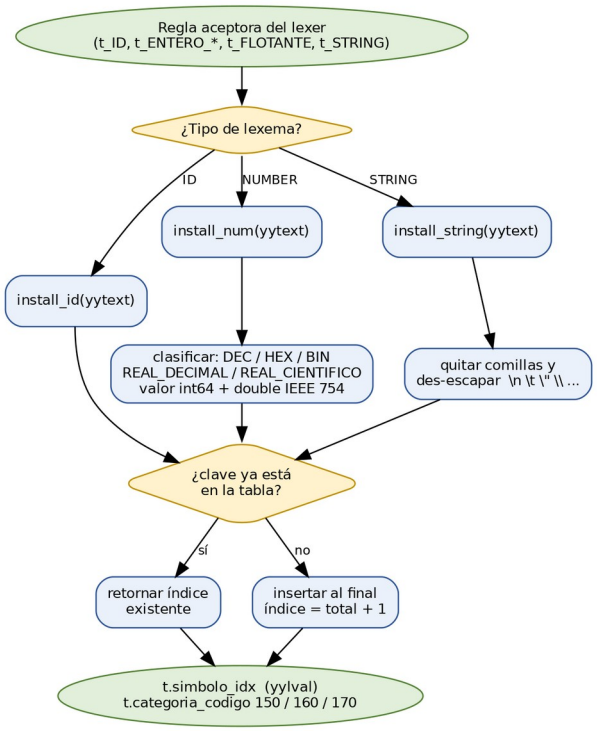


Diagrama 9: Inserción sin duplicados en las tablas ID, NUMBER y STRING.

8. LITERAL 2.E: HERRAMIENTAS, METALENGUAJE Y VERSIONES

Los cuatro módulos se construyeron con Python y lex.py, la alternativa que admite el enunciado; el módulo de tablas de símbolos tiene además una versión en ANSI C. Versiones con las que se ejecutaron las pruebas de este informe:

Componente	Versión	Uso
Python	3.11.16 (entorno conda Enigma-64)	Lenguaje de implementación del SPL y del emulador
PLY (Python Lex-Yacc)	3.11	lex.py: preprocesador, compilador y ensamblador. No viene en la instalación estándar de Python: pip install ply
pytest	9.1.1	Suite de pruebas automatizadas
GCC	11.4.0	Compilación del módulo C de tablas de símbolos (-std=c11 -Wall -Wextra)
GNU Make	del sistema	Makefile de spl/compilador
Tk / tkinter	biblioteca estándar	Interfaz gráfica del emulador (no la usa el SPL)

Las dependencias están declaradas en requirements.txt (pytest>=8.0.0, ply>=3.11) y en environment.yml (python=3.11, tk, pytest==9.1.1, ply>=3.11).

9. MANUAL TÉCNICO

9.1. Instalación del entorno

```
# Opción 1: conda
conda env create -f environment.yml
conda activate Enigma-64

# Opción 2: pip sobre Python 3.10 o superior
pip install -r requirements.txt

# Módulo C de tablas de símbolos (opcional, requiere GCC y make)
cd spl/compilador && make && make test && make clean
```

9.2. Estructura de archivos del SPL

Ruta	Contenido
`spl/preprocesador/lexer.py`	PreprocesadorLexer: reglas lex.py con estados directive y blockcomment
`spl/preprocesador/motor.py`	Preprocesador: macros, inclusión, condicionales y mapa de líneas
`spl/compilador/lexer.py`	CompiladorLexer: vocabulario, operadores, literales y errores
`spl/compilador/tablas.py`	GestorTablasSimbolos con TablaID, TablaNum y TablaString
`spl/compilador/tablas_simbolos.{h,c}, test_tablas.c, Makefile`	Versión ANSI C de las tablas y su batería de pruebas
`spl/ensamblador/lexer.py`	EnsambladorLexer: mnemónicos, registros, directivas y etiquetas
`enigma64/enlazador/objeto.py`	ModuloObjeto: formato .obj, análisis, validación y escritura
`enigma64/enlazador/enlace.py`	EnlazadorEnigma: enlace de dos pasadas y reubicación
`enigma64/cargador.py`	CargadorEnigma y formato ejecutable .e64
`spl/pipeline.py, spl/__main__.py`	OrquestadorSPL y su línea de comandos
`spl/include/enigma64.h`	Cabecera estándar con constantes de la arquitectura
`spl/ejemplos/, spl/compilador/ejemplos/`	Programas de los escenarios de prueba
`tests/test_spl/, tests/core/test_enlazador.py`	Pruebas automatizadas del SPL y del enlazador

9.3. Interfaz de programación

```
from spl.preprocesador import preprocesar_archivo, preprocesar_texto
res = preprocesar_archivo("programa.spl", ["include/"])   # .texto, .macros, .mapa_lineas

from spl.compilador import CompiladorLexer, GestorTablasSimbolos
lexer = CompiladorLexer(tablas=GestorTablasSimbolos())
tokens, errores = lexer.tokenizar(res.texto)
for t in tokens:
    t.type, t.value, t.lineno, t.columna, t.categoria_codigo, t.subcodigo, t.simbolo_idx
print(lexer.tablas.generar_volcado_texto())

from spl.ensamblador import EnsambladorLexer
tokens_asm, errores_asm = EnsambladorLexer().tokenizar(texto_asm)

from enigma64.enlazador import EnlazadorEnigma, analizar_objeto
resultado = EnlazadorEnigma([analizar_objeto("a.obj"),
analizar_objeto("b.obj")]).enlazar(0x00200000, "main")

from spl.pipeline import OrquestadorSPL
informe = OrquestadorSPL().ejecutar_escenario(1).generar_informe()
```

Las clases de los analizadores exponen la tupla tokens que espera ply.yacc, de modo que el analizador sintáctico del Taller # 02 podrá importarla directamente.

9.4. Cómo extender el vocabulario

- **Palabra reservada:** agregarla al diccionario PALABRAS_RESERVADAS_* correspondiente y a su tabla de subcódigos; la regla t_ID la reconoce sin tocar ninguna expresión regular.
- **Operador:** agregar el lexema a OPERADORES_*, el nombre del token a TOKENS_OPERADORES_LITERALES y la regla t_NOMBRE = r'...'; los operadores de varios caracteres deben quedar antes que sus prefijos.
- **Literal:** definir la regla como función antes de la regla de número mal formado e instalar el lexema con el gestor de tablas.
- **Mnemónico:** agregarlo a la tabla de la CPU; la prueba de coherencia exige que el ensamblador lo incluya también.

9.5. Pruebas automatizadas

Archivo	Pruebas	Cobertura
`tests/test_spl/test_preprocesador.py`	45	Macros, inclusión, ciclos, condicionales, comentarios y CLI
`tests/test_spl/test_lexer_compilador.py`	18	Vocabulario, TDA, arreglos, matrices, desambiguación y errores
`tests/test_spl/test_lexer_operadores_literales.py`	104	Operadores, literales, rangos de 64 bits e IEEE 754, escapes y errores
`tests/test_spl/test_tablas.py`	19	InstallID, InstallNum, InstallString y volcado
`tests/test_spl/test_lexer_ensamblador.py`	117	Vocabulario, literales, errores, CLI y los programas .s oficiales
`tests/test_spl/test_pipeline_integracion.py`	11	Escenarios integrales y verificación del módulo C
`tests/core/test_enlazador.py`	50	Formato .obj, marcas, enlace, errores y carga

```
python -m pytest tests/test_spl tests/core/test_enlazador.py -v    # SPL y enlazador
python -m pytest -q                                                # suite completa
```

10. MANUAL DE USUARIO

10.1. Comandos

Todos los comandos se ejecutan desde la raíz del repositorio con el entorno activado:

Comando	Acción
`python -m spl`	Ejecuta los 3 escenarios integrales y la verificación del módulo C
`python -m spl --escenario 1|2|3|todos`	Ejecuta un escenario integral
`python -m spl --spl programa.spl [-I dir] [-D N=V]`	Preprocesa y tokeniza un programa de alto nivel
`python -m spl --asm modulo.asm`	Tokeniza un archivo en ensamblador
`python -m spl --obj a.obj b.obj`	Enlaza, carga y ejecuta módulos objeto
`python -m spl --sin-tablas`	Oculta el volcado de las tablas de símbolos
`python -m spl --verificar-c`	Compila con GCC y ejecuta las pruebas del módulo C
`python -m spl.preprocesador entrada [-o salida] [-I dir] [-D N=V]`	Solo el preprocesador
`python -m spl.compilador [entrada] [--sin-tablas]`	Solo el analizador léxico del compilador (stdin si no hay archivo)
`python -m spl.ensamblador [entrada]`	Solo el analizador léxico del ensamblador
`python -m enigma64.enlazador.ejemplos`	Genera principal.obj y libmat.obj en programas/objetos/
`python -m enigma64.enlazador a.obj b.obj -o demo.e64 --mapa`	Enlaza módulos y escribe el ejecutable; --base, --entrada y --sin-informe son opcionales

El código de salida es 0 si el análisis termina sin errores y 1 en caso contrario, lo que permite usar los comandos en scripts y en integración continua.

10.2. Ejemplo de uso

Tokenizar un fragmento desde la entrada estándar:

```
$ echo 'float area = pi * 2.5e-3;' | python -m spl.compilador --sin-tablas
Línea   Col   Categoría Token     Código  Atributo / Lexema
1       1     FLOAT_TYPE          110     float
1       7     ID                  150     [sym=1] area
1       12    ASIGNAR             230     =
1       14    ID                  150     [sym=2] pi
1       17    POR                 210     *
1       19    FLOTANTE            160     [sym=1] 2.5e-3
1       25    PUNTO_Y_COMA        240     ;
```

El informe del orquestador tiene un bloque por etapa ([ETAPA 1/4] a [ETAPA 4/4]) con métricas, la lista de errores y el estado final de las tablas, y termina con el estado global del pipeline.

10.3. Mensajes de error

Módulo	Mensaje	Causa y corrección
Compilador	`caracter no valido en lenguaje SPL: '@'`	Carácter fuera del alfabeto; eliminarlo
Compilador	`literal numerico mal formado: '0xFFG1'`	Dígito inválido para la base

Módulo	Mensaje	Causa y corrección
Compilador	`literal entero excede el rango de 64 bits con signo`	Decimal mayor que 9223372036854775807
Compilador	`literal hexadecimal excede los 64 bits de Enigma-64`	Más de 16 dígitos hexadecimales significativos
Compilador	`literal flotante desborda / subdesborda ... binary64`	Exponente fuera de rango (1e309, 1e-400)
Compilador	`exponente sin digitos en literal flotante`	Falta el exponente: 2.5e+ → 2.5e+3
Compilador	`secuencia de escape no valida '\q'`	Solo se admiten \n \t \r \0 \\ \" \'
Compilador	`cadena de texto sin cerrar antes del fin de linea`	Falta la comilla de cierre
Compilador	`operador logico incompleto (se esperaba '&&' o '||')`	& o | sueltos
Ensamblador	`registro inexistente (solo R0..R7): 'R8'`	Usar R0–R7 o un alias
Ensamblador	`directiva desconocida: '.float'`	Usar .data .text .org .word .byte
Preprocesador	`ciclo de inclusion detectado: a -> b -> a`	Agregar #pragma once o reorganizar las inclusiones
Enlazador	`SimboloNoDefinido / SimboloDuplicado`	Falta un módulo o dos módulos exportan el mismo nombre

11. EXPERIMENTACIÓN Y ANÁLISIS DE RESULTADOS

11.1. Metodología

Se ejecutaron los tres escenarios integrales del orquestador (python -m spl --escenario N), que recorren las cuatro etapas, y dos escenarios complementarios del analizador del compilador dedicados a operadores y literales (python -m spl.compilador). Todas las cifras de esta sección se tomaron de esas ejecuciones, realizadas el 23 de octubre de 2026 con las versiones de la Sección 8.

11.2. Escenario 1: flujo nominal completo

a. Objetivo: verificar el recorrido completo sin errores: inclusión de enigma64.h, macros, TDA, matriz, while e if-then-else en el compilador; un módulo ensamblador con directivas y direccionamiento indirecto; y el enlace de dos módulos objeto ejecutado en la CPU.

b. Entrada (spl/ejemplos/escenario1_aritmetica_tda.spl):

Archivo `spl/ejemplos/escenario1_aritmetica_tda.spl` (53 líneas, incluido en el repositorio).

Entrada en ensamblador (spl/ejemplos/escenario1_modulo.asm):

Archivo `spl/ejemplos/escenario1_modulo.asm` (22 líneas, incluido en el repositorio).

c. Resultados:

Etapa	Métrica	Valor obtenido
Preprocesador	Líneas de entrada / expandidas	53 / 52
Preprocesador	Archivos procesados / macros activas	2 (fuente y enigma64.h) / 14
Compilador	Tokens / tipos de token distintos	160 / 27
Compilador	tablaID / tablaNum / tablaString	13 / 9 / 0
Compilador	Errores léxicos	0
Ensamblador	Tokens / mnemónicos / registros / directivas / etiquetas	66 / 8 / 15 / 5 / 2
Ensamblador	Errores léxicos	0
Enlazador-Cargador	Módulos / marcas de reubicación resueltas	2 (principal, libmat) / 6

Etapa	Métrica	Valor obtenido
Enlazador-Cargador	Base / punto de entrada / tamaño	0x00200000 / 0x00200000 / 120 bytes
CPU	Instrucciones ejecutadas / R5 (RV)	15 (hasta HLT) / 42
CPU	RAM: resultado / copia (.bss)	42 / 42

d. Estado final de la tabla de constantes numéricas (tablaNum):

Índice	Lexema	Formato	Entero 64 bits	Real IEEE 754
1	`3`	ENTERO_DECIMAL	3	3
2	`16`	ENTERO_DECIMAL	16	16
3	`0`	ENTERO_DECIMAL	0	0
4	`30`	ENTERO_DECIMAL	30	30
5	`12`	ENTERO_DECIMAL	12	12
6	`1.0`	REAL_DECIMAL	1	1
7	`1`	ENTERO_DECIMAL	1	1
8	`2.5`	REAL_DECIMAL	2	2.5
9	`42`	ENTERO_DECIMAL	42	42

e. Análisis:

- **Preprocesador:** las macros DIM_MATRIZ, META_SUMA, VERDADERO y FALSO se sustituyeron antes del análisis léxico: por eso 3 y 42 aparecen en tablaNum aunque el programa escribe nombres. Los comentarios ya no llegan al compilador (0 descartados en esa etapa).
- **Compilador:** los 13 identificadores quedan una sola vez aunque operando_a o indice_fila aparezcan varias veces, y _acumulador confirma el guion bajo como letra inicial. tda y struct se reconocen como palabras reservadas de la categoría 120.
- **Enlace y ejecución:** las 6 marcas (HI16, LO16, ABS32, REL16, ABS32 y ABS64) se resolvieron; la CPU ejecutó 15 instrucciones y dejó 42 en RV y en las variables resultado y copia, el valor esperado.

11.3. Escenario 2: punto flotante, switch-case, cadenas con escapes y condicionales

a. Objetivo: ejercitar literales reales en punto fijo y en notación científica, enteros hexadecimales y binarios, la estructura switch-case, cadenas con escapes, compilación condicional (#ifdef/#else/#endif) y un módulo ensamblador de la FPU.

b. Entrada (spl/ejemplos/escenario2_punto_flotante_switch.spl):

Archivo `spl/ejemplos/escenario2_punto_flotante_switch.spl` (65 líneas, incluido en el repositorio).

c. Resultados:

Etapa	Métrica	Valor obtenido
Preprocesador	Líneas de entrada / expandidas / macros	65 / 56 / 17
Compilador	Tokens / tipos de token distintos	207 / 37
Compilador	tablaID / tablaNum / tablaString	15 / 11 / 2
Compilador	Errores léxicos	0
Ensamblador	Tokens / mnemónicos / registros / directivas / etiquetas	87 / 13 / 17 / 6 / 4
Ensamblador	Errores léxicos	0
Enlazador-Cargador y CPU	Marcas / instrucciones / R5	6 / 15 / 42

d. Tablas de constantes y de cadenas:

Índice	Lexema	Formato	Real IEEE 754
1	`2.0`	REAL_DECIMAL	2
2	`1.0e-9`	REAL_CIENTIFICO	1e-09
3	`0`	ENTERO_DECIMAL	0
4	`64`	ENTERO_DECIMAL	64
5	`1`	ENTERO_DECIMAL	1
6	`1.5E3`	REAL_CIENTIFICO	1500
7	`0.5`	REAL_DECIMAL	0.5
8	`0xFF`	ENTERO_HEX	255
9	`0b1010`	ENTERO_BIN	10
10	`0.0`	REAL_DECIMAL	0
11	`1.0`	REAL_DECIMAL	1

Índice	Texto instalado en tablaString	Longitud
1	`C:\Enigma64\fpu_trace.log`	25
2	`Estado FPU: "CONVERGENCIA_OK"`	29

e. Análisis:

- **Literales:** las cinco formas numéricas quedaron con su formato correcto; 0xFF y 0b1010 se convirtieron a 255 y 10, y 1.0e-9 conserva su valor binary64.
- **Cadenas:** las barras dobles y las comillas escapadas se interpretaron: la ruta queda con barras simples y la segunda cadena contiene las comillas dentro del texto. La longitud cuenta caracteres reales, no los de escape.
- **Condicionales:** como USAR_FPU está definida, MODO_EJECUCION vale 1 y la rama #else se descartó; por eso salen 56 líneas de 65.
- **Hallazgo:** el programa usa la macro con parámetro ESCALAR_DOBLE(x). El preprocesador solo implementa macros de objeto, de modo que la sustituye por el texto literal "(x) ((x) + (x))" seguido de (iter_1_). El resultado sigue siendo léxicamente válido, pero introduce el identificador x en tablaID (índice 13). Las macros con parámetros no son un requisito del taller (incluir y definir lo son) y se documentan en la Sección 12.

11.4. Escenario 3: detección y recuperación de errores léxicos

a. Objetivo: comprobar que cada analizador informa todos los errores con su posición y continúa el análisis, tanto en el compilador como en el ensamblador.

b. Entradas (escenario3_recuperacion_errores.spl y escenario3_errores.asm):

Archivo `spl/ejemplos/escenario3_recuperacion_errores.spl` (26 líneas, incluido en el repositorio).

Archivo `spl/ejemplos/escenario3_errores.asm` (15 líneas, incluido en el repositorio).

c. Errores informados:

Etapa	Línea:columna	Mensaje	Lexema
Compilador	14:13	caracter no valido en lenguaje SPL	`@`
Compilador	15:12	caracter no valido en lenguaje SPL	`$`
Compilador	16:19	literal numerico mal formado	`0b10201`
Compilador	17:15	literal numerico mal formado	`0xFFG1`
Compilador	18:25	literal numerico mal formado	`45xyz`
Ensamblador	9:21	caracter no valido	`@`

Etapa	Línea:columna	Mensaje	Lexema
Ensamblador	10:17	registro inexistente (solo R0..R7)	`R8`
Ensamblador	11:9	directiva desconocida	`.float`
Ensamblador	12:25	literal numerico mal formado	`0b102`

d. Resultados globales:

Etapa	Tokens válidos	Errores	Tablas ID / NUMBER / STRING
Compilador	54	5	9 / 5 / 0
Ensamblador	35	4	—
Enlazador-Cargador	2 módulos, 6 marcas	0	Enlazado sin ejecutar la CPU

e. Análisis:

- **Recuperación:** tras cada error se reconocieron los tokens siguientes de la misma línea (por ejemplo invalida, = y 10 después de @) y la función verificar_recuperacion se tokenizó completa; el estado final del pipeline es «exitoso con recuperación de errores».
- **Literales mal formados:** 0b10201, 0xFFG1 y 45xyz se informaron como un solo error cada uno, en vez de partirse en un número válido seguido de basura, gracias al contexto derecho FIN_NUM.
- **Límite de la fase léxica:** MOV no pertenece al ISA, pero tiene forma de identificador y el ensamblador lo clasifica como ETIQUETA_REF; detectar que una etiqueta aparece donde se espera un mnemónico es tarea del análisis sintáctico.
- **Numeración:** las líneas del compilador corresponden al código expandido por el preprocesador: el carácter @ está en la línea 12 del archivo fuente y se informa en la 14. Las del ensamblador coinciden con el archivo porque este no pasa por el preprocesador (Sección 12).

11.5. Escenario complementario 4: operadores y literales válidos

a. Objetivo: cubrir todos los operadores aritméticos, relacionales y lógicos y todas las formas de literal, incluidos los valores límite de 64 bits, sin errores.

b. Entrada (spl/compilador/ejemplos/escenario4_operadores_literales.spl):

Archivo `spl/compilador/ejemplos/escenario4_operadores_literales.spl` (43 líneas, incluido en el repositorio).

c. Resultados: 182 tokens, 0 errores léxicos; tablaID con 23 identificadores, tablaNum con 11 constantes y tablaString con 2 cadenas.

Lexema	Formato	Valor almacenado
`0X7FFFFFFFFFFFFFFF`	ENTERO_HEX	9223372036854775807
`9223372036854775807`	ENTERO_DECIMAL	9223372036854775807 (INT64_MAX, aceptado)
`.5 / 5.`	REAL_DECIMAL	0.5 / 5
`6.022e23`	REAL_CIENTIFICO	6.022e+23
`2.5E-3 / .5e+2`	REAL_CIENTIFICO	0.0025 / 50
`"Rango valido:\t[0, 1000]\n"`	STRING	texto con tabulador y salto reales, longitud 24
`"Ruta: \"C:\\Enigma64\\ salida.log\""`	STRING	Ruta: "C:\Enigma64\salida.log", longitud 30

Análisis: los operadores de dos caracteres (==, !=, <=, >=, &&, ||) nunca se partieron, y ! seguido de (se reconoció como NOT_LOGICO. Los extremos .5 y 5. de la mantisa y el exponente con signo explícito se aceptan, y el mayor entero decimal representable se instala sin error.

11.6. Escenario complementario 5: errores en operadores y literales

a. Objetivo: verificar los errores propios de los literales: desbordamientos de 64 bits y de binary64, exponentes incompletos, escapes no admitidos, operadores lógicos incompletos y cadenas sin cerrar.

b. Entrada (spl/compilador/ejemplos/escenario5_errores_literales.spl):

Archivo `spl/compilador/ejemplos/escenario5_errores_literales.spl` (31 líneas, incluido en el repositorio).

c. Errores informados (9) y resultado: 50 tokens válidos, 11 comentarios ignorados, 31 líneas analizadas.

Línea:columna	Mensaje	Lexema
9:20	literal entero excede el rango de 64 bits con signo	`9223372036854775808`
10:20	literal hexadecimal excede los 64 bits de Enigma-64	`0x10000000000000000`
13:18	literal flotante desborda el rango de IEEE 754 binary64	`1e309`
14:20	literal flotante subdesborda a cero en IEEE 754 binary64	`1e-400`
17:19	exponente sin digitos en literal flotante	`2.5e+`
20:12	secuencia de escape no valida '\q' en cadena	`"Escape invalido: \q"`
23:30	operador logico incompleto (se esperaba '&&' o '||')	`&`
23:34	operador logico incompleto (se esperaba '&&' o '||')	`|`
26:12	cadena de texto sin cerrar antes del fin de linea	`"Sin comillas de cierre);`

Análisis: 9223372036854775808 es exactamente INT64_MAX + 1, y 0x10000000000000000 es el primer valor que necesita 65 bits, así que el límite se detecta en el primer valor que lo excede. 1e309 y 1e-400 son literales sintácticamente correctos cuyo valor binary64 sería inf y 0; rechazarlos evita que la constante instalada difiera de la escrita. Después de cada error se reconocen los tokens restantes, incluida la función final del archivo.

11.7. Suite de pruebas automatizadas

La suite completa del repositorio (emulador, FPU, SPL y enlazador) terminó con 835 pruebas aprobadas, 1 omitida a propósito y 308 subpruebas aprobadas. Las 364 pruebas del SPL y del enlazador listadas en la Sección 9.5 aprobaron todas. El módulo C compiló con GCC 11.4.0 sin advertencias y su batería de pruebas terminó con código 0.

11.8. Análisis global

- **Correctitud:** en los escenarios sin errores, todos los tokens se clasificaron en la categoría esperada y las tablas no tienen duplicados; el valor final de la ejecución (42) coincide con el esperado.
- **Robustez:** en los escenarios con errores, cada error se informó una vez, con posición y lexema, y el análisis siguió hasta el fin del archivo.
- **Coherencia con la máquina:** los límites de los literales son exactamente los de los registros de 64 bits y del formato binary64 de la FPU, y el ensamblador reconoce exactamente las 41 instrucciones que decodifica la CPU.

12. LIMITACIONES CONOCIDAS Y TRABAJO PARA EL TALLER # 02

La revisión del sistema frente al enunciado y los escenarios de prueba dejan los siguientes puntos pendientes:

Punto	Situación actual	Trabajo previsto
Ensamblador (2.b)	Implementa la fase léxica. La traducción de .asm a módulo .obj aún no existe: enigma64/ensamblador.py produce bytes planos y no emite marcas de reubicación.	Analizador sintáctico con ply.yacc y emisión de .obj con las marcas de la Sección 6.2.
Encadenamiento del pipeline	Cada etapa recibe su propia entrada: el escenario integral entrega al ensamblador un .asm escrito a mano y al enlazador los módulos de ejemplo principal.obj y libmat.obj, no la salida de la etapa anterior.	Conectar compilador → ensamblador → enlazador cuando existan la generación de código y la emisión de .obj.
Análisis del .obj (2.c)	El formato es binario y se analiza con registros de tamaño fijo (struct), no con reglas de lex.py.	Si se exige el metalenguaje también aquí, definir una representación textual del módulo objeto ([MODULE], [CODE], [DATA], [SYMBOLS], [EXTERNAL]) con su lexer.
Macros con parámetros	#define F(x) ... se trata como macro de objeto (Escenario 2).	Reconocer la lista de parámetros y sustituir los argumentos.
Numeración de líneas	Los errores del compilador se informan con las líneas del código expandido.	Traducir la posición con el mapa de líneas que ya devuelve el preprocesador (mapa_lineas).
Valor entero de reales grandes	En el volcado de tablaNum, la columna entera de 6.022e23 muestra un valor que no cabe en 64 bits; el valor real es correcto.	Acotar esa columna o mostrarla vacía para los reales.

13. CONCLUSIONES

- El taller dota a Enigma-64 de la primera fase de su sistema de procesamiento del lenguaje: un preprocesador con inclusión, definición de macros y compilación condicional; los analizadores léxicos del compilador y del ensamblador; y un enlazador-cargador que lleva módulos objeto hasta la ejecución en la CPU emulada.
- El vocabulario del compilador (13 palabras de control, 6 de tipos, 4 de TDA, 12 delimitadores, 14 operadores y 5 formas de literal) se organizó en 11 categorías gramaticales con códigos de 100 a 240 y se especificó con definiciones regulares y extensiones (clases, cerraduras, opcionalidad y contexto derecho).
- Usar lex.py exigió diseñar el orden de las reglas, porque PLY no aplica la coincidencia más larga entre reglas distintas. Tratar las palabras reservadas con una sola regla de identificador y ubicar las reglas de error justo después de las válidas reprodujo el comportamiento de FLEX.
- Vincular los literales a la máquina, con enteros de 64 bits y reales binary64, permite detectar en la fase léxica desbordamientos que de otro modo producirían constantes distintas de las escritas.
- Los tres escenarios integrales, los dos complementarios y 364 pruebas automatizadas del SPL y del enlazador confirman el comportamiento descrito, incluida la recuperación ante errores. La revisión también identificó los puntos de la Sección 12, que orientan el Taller # 02.

REFERENCIAS

- Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). Compilers: Principles, Techniques, and Tools (2.ª ed.). Addison-Wesley. Capítulos 1 y 3.
- Levine, J. (2009). flex & bison. O'Reilly Media.
- Paxson, V. et al. Lexical Analysis With Flex, manual de GNU flex 2.6.
- Beazley, D. M. PLY (Python Lex-Yacc), documentación de la versión 3.11. https://www.dabeaz.com/ply/

- Beck, L. L. (1997). System Software: An Introduction to Systems Programming (3.ª ed.). Addison-Wesley. Capítulos 2 y 3 (ensambladores, enlazadores y cargadores).
- IEEE (2019). IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2019.
- Grupo A (2026). Tarea 9: Diseño de un computador propio (Enigma-64, Noctua Systems). Lenguajes de Programación, Universidad Nacional de Colombia.
- Grupo A (2026). Tarea 10: Emulación de su computador Enigma-64. Lenguajes de Programación, Universidad Nacional de Colombia.

ANEXO A. INVENTARIO DEL CÓDIGO FUENTE

El código fuente completo se entrega junto con este documento (repositorio Enigma-64 del grupo). La tabla lista los archivos que componen el SPL, con su número de líneas; los patrones de cada analizador se transcriben en las Secciones 4, 5 y 7.

Archivo	Líneas	Contenido
`spl/preprocesador/lexer.py`	262	Analizador léxico del preprocesador (lex.py)
`spl/preprocesador/motor.py`	567	Motor del preprocesador
`spl/preprocesador/__main__.py`	77	Línea de comandos del preprocesador
`spl/compilador/lexer.py`	622	Analizador léxico del compilador (lex.py)
`spl/compilador/tablas.py`	387	Tablas de símbolos en Python
`spl/compilador/tablas_simbolos.h`	149	Tablas de símbolos en ANSI C: interfaz
`spl/compilador/tablas_simbolos.c`	385	Tablas de símbolos en ANSI C: implementación
`spl/compilador/test_tablas.c`	200	Pruebas del módulo C
`spl/compilador/Makefile`	25	Compilación del módulo C
`spl/compilador/__main__.py`	84	Línea de comandos del compilador
`spl/ensamblador/lexer.py`	277	Analizador léxico del ensamblador (lex.py)
`spl/ensamblador/__main__.py`	67	Línea de comandos del ensamblador
`enigma64/enlazador/objeto.py`	930	Formato y análisis de módulos .obj
`enigma64/enlazador/enlace.py`	451	Enlazador de dos pasadas
`enigma64/enlazador/ejemplos.py`	173	Módulos objeto de ejemplo
`enigma64/enlazador/__main__.py`	76	Línea de comandos del enlazador
`enigma64/cargador.py`	861	Cargador y formato .e64
`spl/pipeline.py`	518	Orquestador del SPL
`spl/__main__.py`	145	Línea de comandos del orquestador
`spl/include/enigma64.h`	15	Cabecera estándar